

SISTEMA DE INVENTARIO Y CONTROL DE KARDEX

Manual Técnico de Arquitectura, Flujos y Código Fuente

Documento descriptivo sobre el funcionamiento lógico, arquitectura de datos, transaccionalidad del Kardex y flujo de control de inventarios. Preparado para la sustentación de tesis y referencia técnica del sistema.

TECNOLOGÍAS CORE:

- Backend: Python + FastAPI + SQLAlchemy
- Frontend: Angular + Angular Material
- Base de Datos: PostgreSQL

Generado el: 14/06/2026

Estado del Proyecto: Producción / Listo para Despliegue

1. ARQUITECTURA GENERAL DEL SISTEMA

El sistema está diseñado bajo una arquitectura limpia y desacoplada de dos capas principales: un Backend API REST y un Frontend SPA (Single Page Application).

1.1 Tecnologías Utilizadas en el Backend

- Python y FastAPI: Framework web de alto rendimiento basado en tipado estático, asincronía y generación automática de documentación OpenAPIs (Swagger UI).
- SQLAlchemy: ORM (Object-Relational Mapping) utilizado para interactuar con la base de datos de manera orientada a objetos. Proporciona control preciso de transacciones y sentencias SQL optimizadas.
- Alembic: Herramienta de gestión de migraciones de base de datos, garantizando la consistencia del esquema a lo largo del desarrollo y en producción.
- PostgreSQL: Motor de base de datos relacional robusto que soporta restricciones avanzadas, índices eficientes y transaccionalidad ACID absoluta.

1.2 Estructura del Directorio Backend

```
servidor/  
|-- alembic/                # Migraciones y control de versiones del esquema BD  
|-- aplicacion/            # Carpeta raíz del código de la API  
|   |-- api/               # Capa de controladores y rutas (endpoints)  
|   |   |-- v1/            # Rutas agrupadas de la versión 1 del API  
|   |-- esquemas/          # Clases Pydantic para validación de entrada/salida  
|   |-- modelos/           # Clases SQLAlchemy que representan las tablas en BD  
|   |-- nucleo/            # Archivos de configuración general, seguridad y BD  
|   |-- servicios/         # Capa de lógica de negocio (Servicios del sistema)  
|   |-- principal.py        # Punto de entrada de la aplicación FastAPI  
|-- pruebas/               # Suite de pruebas automatizadas (con Pytest)  
|-- requirements.txt        # Dependencias de librerías del backend  
-- .env                    # Variables de entorno críticas de configuración
```

1.3 Tecnologías Utilizadas en el Frontend

La interfaz del sistema está construida en Angular y Angular Material. Sigue una estructura modular organizada en español para alinearse al negocio, dividida por entidades del dominio (Productos, Categorías, Kardex, Clientes, Proveedores, Usuarios, Tablero). Usa el enrutamiento nativo de Angular y guards para la protección de rutas basada en roles de usuarios.

2. MODELO DE DATOS Y RESTRICCIONES DE INTEGRIDAD

El diseño de base de datos prioriza la consistencia e integridad referencial. Para evitar estados de stock inconsistentes o registros huérfanos, se implementan restricciones (Constraints) a nivel de motor de base de datos.

2.1 Modelos Principales (SQLAlchemy)

1. Producto (tabla 'productos'): Define el código único, nombre, descripción, precio de venta, stocks actual y mínimo, y sus relaciones con categoría y proveedor.
2. Movimiento (tabla 'movimientos'): Representa una transacción en el Kardex. Registra el tipo (entrada, salida, ajuste), la cantidad, el costo unitario, el stock anterior e inmediatamente posterior, la referencia, el motivo de la acción y el usuario que la ejecutó.
3. Categoría ('categorias'), Cliente ('clientes'), Proveedor ('proveedores') y Usuario ('usuarios'): Entidades complementarias indispensables para el flujo de trazabilidad.

2.2 Restricciones de Integridad del Kardex

En el archivo modelo 'movimiento.py', se configuran CheckConstraints nativas de SQLAlchemy para proteger la base de datos contra inserciones corruptas o incorrectas:

```
class Movimiento(Base):
```

```
__tablename__ = 'movimientos'
__table_args__ = (
    # Evita cantidades negativas o cero en transacciones
    CheckConstraint('cantidad > 0', name='ck_movimientos_cantidad_positiva'),
    # Restringe los tipos a los tres válidos del negocio
    CheckConstraint(
        "tipo IN ('entrada', 'salida', 'ajuste')",
        name='ck_movimientos_tipo_valido',
    ),
    # Impide costos unitarios negativos
    CheckConstraint(
        'costo_unitario IS NULL OR costo_unitario >= 0',
        name='ck_movimientos_costo_unitario_no_negativo',
    ),
)
```

[NOTA] Estas restricciones son aplicadas directamente en PostgreSQL por Alembic, asegurando que ningún proceso externo pueda violar las reglas de integridad de negocio.

3. FLUJO LÓGICO DE PRODUCTOS Y INVENTARIO

La gestión de productos maneja los datos maestros de los ítems del almacén. El flujo de control garantiza integridad en operaciones CRUD y previene la eliminación de productos con transacciones existentes.

3.1 Lógica de Creación y Validación de Productos

Al registrar un producto, el sistema valida que el tipo ('repuesto', 'producto', 'insumo') sea correcto, que la categoría exista y asocie opcionalmente un proveedor. Además, maneja de forma asíncrona la subida de imágenes guardándolas en un almacenamiento local físico del servidor y retornando la URL pública estructurada.

3.2 Lógica de Eliminación Segura (Prevenir Pérdida de Trazabilidad)

No está permitido eliminar un producto si ya tiene movimientos en el Kardex. De lo contrario, se destruiría la consistencia histórica de los inventarios. La función 'eliminar_producto' valida esto contando las transacciones del Kardex antes de proceder a borrar el registro e hilos de imágenes:

```
def eliminar_producto(db: Session, producto_id: int) -> None:
    p = require_producto(db, producto_id)
    # Cuenta movimientos relacionados al producto
    count = db.query(func.count(Movimiento.id)).filter(
        Movimiento.producto_id == producto_id
    ).scalar()
    if count and int(count) > 0:
        raise ProductoEnUsoError() # Lanza error HTTP 400 prohibiendo el borrado
    image_url = p.image_url
    db.delete(p)
    db.commit()
    delete_producto_image(image_url) # Elimina archivo del servidor
```

4. EL MÓDULO KARDEX: MOVIMIENTOS Y CONTROL DE STOCK

El Kardex es el corazón del sistema, permitiendo la trazabilidad total de entradas, salidas y ajustes de inventario. Un movimiento modifica el stock actual del producto en la tabla 'productos' y crea un registro histórico en la tabla 'movimientos'.

4.1 Reglas del Negocio para cada Tipo de Movimiento

1. ENTRADA (Ingreso): Suma la cantidad solicitada al stock actual del producto. Asocia opcionalmente un Proveedor.
2. SALIDA (Despacho): Resta la cantidad al stock actual, validando que exista suficiente disponibilidad. Asocia opcionalmente un Cliente.
3. AJUSTE (Inventario físico): Reemplaza directamente el stock actual por la cantidad ingresada, útil para regularizaciones físicas. No asocia tercero.

4.2 Actualizaciones de Stock Concurrentes y Seguras

Para evitar condiciones de carrera (Race Conditions) y sobreescrituras en sistemas concurrentes con múltiples usuarios vendiendo al mismo tiempo, el backend actualiza el stock ejecutando sentencias de actualización directamente calculadas en la base de datos (Database-level locks/atomic updates) usando SQLAlchemy 'sa_update' y retornando el valor final mediante 'returning()'.

4.3 Código de Actualización Atómica y Creación de Movimiento

A continuación se detalla el código implementado en 'servicio_movimiento.py' que aplica los cambios y calcula el Kardex:

```
def _aplicar_stock_entrada(db: Session, producto_id: int, cantidad: int) -> tuple[int, int]:
    result = db.execute(
        sa_update(Producto)
        .where(Producto.id == producto_id)
        .values(stock_actual=Producto.stock_actual + cantidad)
        .returning(Producto.stock_actual)
    ).scalar_one_or_none()
    if result is None: raise ProductoNoEncontradoError()
    stock_posterior = int(result)
    return stock_posterior - cantidad, stock_posterior

def _aplicar_stock_salida(db: Session, producto_id: int, cantidad: int) -> tuple[int, int]:
    # Importante: Garantiza stock suficiente atómicamente en el filtro de la consulta
    result = db.execute(
        sa_update(Producto)
        .where(Producto.id == producto_id, Producto.stock_actual >= cantidad)
        .values(stock_actual=Producto.stock_actual - cantidad)
        .returning(Producto.stock_actual)
    ).scalar_one_or_none()
    if result is None:
        if db.query(Producto.id).filter(Producto.id == producto_id).first() is None:
            raise ProductoNoEncontradoError()
        raise StockInsuficienteError() # Lanza HTTP 400
    stock_posterior = int(result)
    return stock_posterior + cantidad, stock_posterior
```

4.4 La Función 'registrar_movimiento'

Esta función orquesta la validación, invoca la lógica de stock, determina relaciones (cliente/proveedor) y escribe en el Kardex en un bloque protegido contra excepciones que ejecuta rollback si algo falla:

```
def registrar_movimiento(db: Session, usuario_id: int, payload: MovimientoCreate) -> Movimiento:
    try:
        if payload.cantidad <= 0:
            raise CantidadInvalidaError()
        # Lógica de stock según tipo de movimiento...
        if payload.tipo == 'entrada':
            stock_anterior, stock_posterior = _aplicar_stock_entrada(db, payload.producto_id, payload.cantidad)
        elif payload.tipo == 'salida':
            stock_anterior, stock_posterior = _aplicar_stock_salida(db, payload.producto_id, payload.cantidad)
        elif payload.tipo == 'ajuste':
            stock_anterior, stock_posterior = _aplicar_stock_ajuste(db, payload.producto_id, payload.cantidad)
        else:
            raise TipoMovimientoInvalidoError()

        cliente_id = payload.cliente_id if payload.tipo == 'salida' else None
        proveedor_id = payload.proveedor_id if payload.tipo == 'entrada' else None

        movimiento = Movimiento(
            producto_id=payload.producto_id, usuario_id=usuario_id, cliente_id=cliente_id,
            proveedor_id=proveedor_id, tipo=payload.tipo, cantidad=payload.cantidad,
            costo_unitario=payload.costo_unitario, referencia=payload.referencia, motivo=payload.motivo,
            observacion=payload.observacion, stock_anterior=stock_anterior, stock_posterior=stock_posterior
        )
        db.add(movimiento)
        db.flush() # Envía los cambios preliminares a base de datos
        db.commit() # Confirma la transacción en bloque único
        db.refresh(movimiento)
        return movimiento
    except Exception:
        db.rollback() # Revierte todo el bloque en caso de fallo en BD o código
        raise
```

5. SEGURIDAD Y AUTENTICACIÓN DEL SISTEMA

El sistema resguarda los endpoints sensibles mediante autenticación basada en tokens JWT (JSON Web Tokens) y contraseñas encriptadas con algoritmos de hashing seguro (bcrypt).

5.1 Encriptación de Contraseñas y Generación de Tokens

Al registrar un nuevo usuario, su contraseña se procesa a través del método 'hash_password' de bcrypt. La base de datos almacena únicamente el hash resultante ('hashed_password') y nunca la contraseña en texto plano. Durante el inicio de sesión, se verifica el hash y se genera el token JWT que viaja en la cabecera 'Authorization' de los requests subsiguientes.

5.2 Autorización Basada en Roles (Roles de Administrador y Usuario)

El sistema admite dos roles: 'admin' (administrador) y 'usuario' (operador general).

- Operador General ('usuario'): Puede listar productos, registrar movimientos (entradas/salidas/ajustes) y revisar su Kardex personal y consultas al chatbot.
- Administrador ('admin'): Único rol facultado para crear/actualizar productos, gestionar categorías, crear otros usuarios y acceder a reportes de exportación.

6. REPORTES Y EXPORTACIÓN DE DATOS

El sistema cuenta con motores integrados para generar reportes dinámicos basados en los filtros de búsqueda especificados por el usuario.

6.1 Exportación a Excel (Librería openpyxl)

Construye hojas de cálculo Excel (.xlsx) estructuradas en memoria usando un búfer binario (BytesIO). Aplica estilos corporativos personalizados, tablas con autofiltros, inmovilización de paneles y celdas con formatos numéricos específicos (ej. monedas '\$/ #,##0.00' y fechas 'yyyy-mm-dd hh:mm').

6.2 Exportación a PDF (Librería fpdf2)

Genera documentos en formato PDF vectoriales directamente desde el servidor. El diseño cuenta con:

- Cabecera del reporte con filtros aplicados e información del inventario.
- Tabla de movimientos formateada con colores diferenciados para cada tipo de transacción (ej. Entradas en verde, Salidas en rojo y Ajustes en morado).
- Numeración de páginas automática en el pie de página.

```
# Fragmento de lógica de coloración en PDF (servicio_exportacion.py)
for i, cell in enumerate(line):
    if i == 4: # Columna 'Tipo'
        if dto.tipo == 'entrada':
            pdf.set_text_color(22, 163, 74) # Verde
        elif dto.tipo == 'salida':
            pdf.set_text_color(220, 38, 38) # Rojo
        elif dto.tipo == 'ajuste':
            pdf.set_text_color(124, 58, 237) # Morado
    else:
        pdf.set_text_color(15, 23, 42) # Texto oscuro por defecto
    pdf.cell(col_w[i], 6, _pdf_safe(cell), border=1, fill=fill)
```

7. CHATBOT INTELIGENTE DE INVENTARIO

Una de las características más avanzadas del sistema es el Chatbot integrado ('servicio_chatbot.py'). Este componente permite a los usuarios interactuar con el inventario mediante lenguaje natural, facilitando consultas rápidas sin navegar por las opciones tradicionales.

7.1 Clasificación de Intenciones por Reglas y Fallback de IA

El chatbot procesa los mensajes del usuario a través de dos mecanismos principales:

1. Clasificación Basada en Reglas: Analiza el texto normalizado buscando patrones específicos y palabras clave ('ayuda', 'kardex', 'bajo stock', 'valor del inventario', 'top vendidos', 'más caro'). Es ultra rápido y no genera costos adicionales.
2. Clasificación por IA Externa (Fallback): Si está habilitado (AI_ENABLED=true) y las reglas de negocio no son concluyentes, realiza un request estructurado a la API de OpenAI (GPT) para identificar el intent del mensaje del usuario de manera semántica.

7.2 Extracción de Entidades y Contexto de Sesión

El servicio cuenta con un mecanismo que recuerda el producto de la última pregunta (Contexto de Sesión). Si el usuario pregunta: ¿Cuánto stock tiene el Filtro de Aceite?, y a continuación dice: ¿Cuál es su precio?, el chatbot asocia automáticamente la palabra 'su' con el producto Filtro de Aceite guardado en la memoria de sesión temporal.

8. FLUJO Y ARRANQUE DE LA APLICACIÓN

El ciclo de vida del backend incluye una fase de inicialización ('Lifespan') crucial. Al arrancar el servidor FastAPI, se lee el archivo de configuración y se ejecutan de forma automática las tareas de puesta en marcha:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info('Iniciando datos de prueba...')
    db = SessionLocal()
    try:
        # 1. Semilla del administrador (Se crea si no existe o se actualiza)
        seed_admin_user(db, ...)
        # 2. Carga automática de datos de prueba para demostración inmediata
        bootstrap_demo_data(db)
        logger.info('Datos de prueba inicializados correctamente!')
    except Exception as e:
        logger.error(f'Error al inicializar datos: {e}')
    finally:
        db.close()
    yield # El servidor queda activo escuchando peticiones HTTP
```

[NOTA] Esto garantiza que al desplegar el proyecto en un entorno vacío (como Docker o Neon Cloud), el sistema cuente con un usuario administrador funcional y registros de prueba para ser validado inmediatamente por auditores o docentes.